

Thread ի՞նչ է — Խորը ուղեցույց

C++ Concurrency presentation

Այս ուղեցույցը մանրամասն բացատրում է thread-ը՝ ինչ է, process vs thread, ինչպես ստեղծել (std::thread), join/detach, օրինակներով և հարցազրույցի հարցերով:

1. Ի՞նչ է Thread-ը

Thread-ը («թելիկ») ծրագրի **ամենափոքր կատարողական միավոր** է: Մեկ process-ի մեջ կարող են լինել **մի քանի thread**, որ աշխատում են զուգահեռ (parallel/concurrent):

Process = աշխատող ծրագիր (ունի իր memory space)

Thread = process-ի մեջ կատարողական հոսք (կիսում է process-ի memory-ն)

```
#include <thread>
#include <iostream>

void task() { std::cout << "Hello from thread\n"; }

int main() {
    std::thread t(task);    // նոր thread ստեղծում
    t.join();               // սպասում մինչև t ավարտվի
}
```

2. Process vs Thread

PROCESS:

- Ունի անկախ memory space
- Ծանր (ստեղծումը թանկ)
- Մեկը մյուսին չի ազդում (isolated)
- Communication: IPC (pipes, sockets)

THREAD:

- Կիսովի՛ս է process-ի memory-ն (shared)
- Թեթև (ստեղծումը էժան)
- Մեկ thread-ի սխալը կարող է տապալել ամբողջը
- Communication: shared memory (ուղիղ)

	Process	Thread
Memory	առանձին	կիսված (shared)
Ստեղծում	թանկ	էժան
Isolation	այո	ոչ
Communication	IPC	shared memory
Crash-ի ազդեցություն	isolated	ամբողջը

3. Thread ստեղծում (std::thread)

Function-ով

```
void work(int id) { cout << "Thread " << id << "\n"; }
std::thread t(work, 1); // argument-ներ
t.join();
```

Lambda-ով

```
std::thread t([]() { cout << "lambda thread\n"; });
t.join();
```

Մի քանի thread

```
std::vector<std::thread> threads;
for (int i = 0; i < 5; i++)
    threads.emplace_back([i]() { cout << "Thread " << i << "\n"; });
for (auto& t : threads) t.join();
```

4. join() vs detach()

```
std::thread t(task);
t.join();      // սպասում է մինչև thread-ը ավարտվի (blocking)
```

```
std::thread t(task);
t.detach();    // thread-ը աշխատում է անկախ (background), չենք սպասում
```

join() -> սպասում ավարտին (սովորաբար սա)
detach() -> անկախ (daemon-ի պես), չենք կարող join անել հետո

Կարևոր. thread object-ը ոչնչանալուց առաջ պետք է կամ `join()` կամ `detach()` կանչել, հակառակ դեպքում -> `std::terminate` (crash):

5. Thread-ի վտանգները (ներհատուկ)

Քանի որ thread-ները կիսում են memory-ն, առաջանում են խնդիրներ:

Race Condition	-> մի քանի thread միասին փոխում են նույն data
Deadlock	-> thread-ներ սպասում են իրար -> կախված
Data Race	-> synchronization-ից առանց shared data մուտք -> UB

Սրանք լուծվում են `mutex`, `lock_guard`, `atomic` -ով (տես առանձին presentation-ներ):

6. Ինչու Thread-ներ

- + Parallelism (մի քանի CPU core -> ավելի արագ)
- + Responsiveness (UI չի սառնում երկար operation-ի ժամանակ)
- + Resource sharing (սովորաբար memory -> արագ communication)
- Complexity (race condition, deadlock)
- Debugging դժվար (non-deterministic)

7. Հարցազրույցի հարցեր և պատասխաններ

Thread ի՞նչ է:

Ծրագրի ամենափոքր կատարողական միավոր; մեկ process-ի մեջ մի քանի thread, կիսում են memory-ն:

Process vs Thread:

Process' առանձին memory, ծանր, isolated. Thread' shared memory, թեթև, մեկ process-ի մեջ:

join vs detach:

join սպասում է thread-ի ավարտին; detach thread-ը դառնում անկախ (background):

Ինչո՞ւ պետք է join/detach:

thread object ոչնչանալուց առաջ; հակառակ դեպքում std::terminate:

Thread-ի հիմնական խնդիրները:

Race condition, deadlock, data race (shared memory-ի պատճառով):

Thread ինչի՞ համար:

Parallelism (multi-core), responsiveness, resource sharing:

Ամփոփում (Cheat Sheet)

THREAD = ծրագրի ամենափոքր կատարողական միավոր, կիսում process-ի memory

PROCESS vs THREAD:

process -> առանձին memory, ծանր, isolated, IPC

thread -> shared memory, թեթև, մեկ process, shared-memory communication

`std::thread t(func, args) | t.join()` (սպասում) | `t.detach()` (անկախ)

նշանակալուց առաջ -> join/detach (այլպես `std::terminate`)

ԽՆԴԻՐՆԵՐ: race condition | deadlock | data race -> mutex/atomic

ԿԻՐԱՌ: parallelism | responsiveness | resource sharing